

AN INTERACTIVE JAVA TECHNICAL REVIEW SIMULATOR

A PROJECT BASED LEARNING (PBL) REPORT

Submitted

by

RITHIKA S (2104251040779)

SHWETHAA JV (2104251050318)

Submitted in partial fulfilment of the requirements

for the

Project-Based Learning component of Java Programming

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



CHENNAI INSTITUTE OF TECHNOLOGY



(Autonomous)

Affiliated to Anna University, Chennai

October - 2026

Vision of the Institute:

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

Mission of the Institute:

- IM1.** To create next generation leaders by effective teaching learning methodologies and in still Scientifics park in them to meet the global challenges.
- IM2.** To transform lives through deployment of emerging technology, novelty and sustainability.
- IM3.** To inculcate human values and ethical principles to cater the societal needs.
- IM4.** To contribute towards the research ecosystem by providing a suitable, effective platform for interaction between industry, academia and R&D establishments.
- IM5:** To nurture incubation centers enabling structured entrepreneurship and start-ups

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision of the Department:

To Excel in the emerging areas of Computer Science and Engineering by imparting knowledge, relevant practices and inculcating human values to transform the students as potential resources to contribute innovatively through advanced computing in real time situations.

Mission of the Department:

- DM1.** To provide strong fundamentals and technical skills for Computer Science applications through effective teaching learning methodologies.
- DM2.** To transform lives of the students by nurturing ethical values, creativity and novelty to become Entrepreneurs and establish start-ups.
- DM3.** To habituate the students to focus on sustainable solutions to improve the quality of life and the welfare of the society.
- DM4:** To enhance the fabric of research in computing through collaborative linkages with industry and academia.
- DM5:** To inculcate learning of the emerging technologies to pursue higher studies

PROGRAM EDUCATIONAL OBJECTIVES (PEO):

PEO 1: Contribute to the industry as an Engineer through sound knowledge acquired in core engineering to develop new processes and implement the solutions for industrial problems.

PEO 2: Establish an organization / industry as an Entrepreneur with professionalism, leadership quality, teamwork, and ethical values to meet the societal needs.

PEO 3: Create a better future by pursuing higher education / research and develop the sustainable products / solutions to meet the demand.

PROGRAM OUTCOMES (POS):

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for
i) independent and life-long learning ii) adaptability to new and emerging technologies and
iii) critical thinking in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOS):

PSO 1 : Analyze, design and develop solutions in the areas of Business Process Management to build the quality products for industry and social needs.

PSO 2 : Innovate ideas and solutions for real time problems in the field of Software Engineering and Mobile applications by adapting the emerging technologies and tools.

CHENNAI INSTITUTE OF TECHNOLOGY, CHENNAI

(Autonomous)

Affiliated to Anna University, Chennai

BONAFIDE CERTIFICATE

This is to certify that the Project-Based Learning report titled “**AN INTERACTIVE JAVA TECHNICAL REVIEW SIMULATOR**” is a Bonafide record of work carried out by **RITHIKA S (2104251040779)** and **SHWETHAA (2104251050318)** of the Department of Computer Science and Engineering, Chennai Institute of Technology, as part of the continuous, mentor-guided Project-Based Learning (PBL) component of the Java Programming course during the academic year 2026–2027 under my supervision.

Submitted for the final review held on

SIGNATURE

Dr. S. Pavithra M.E., Ph. D

HEAD OF THE DEPARTMENT

Professor and Head,
Department of Computer Science and
Engineering
Chennai Institute of Technology
Chennai - 69

SIGNATURE

Dr. R. Kavitha M.E., Ph. D

SUPERVISOR

Associate Professor
Department of Computer Science
and Engineering
Chennai Institute of Technology
Chennai - 69

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We jointly declare that the PBL report on “**AN INTERACTIVE JAVA TECHNICAL REVIEW SIMULATOR**” is the result of original work done by us and best of our knowledge, similar work has not been submitted to “**ANNA UNIVERSITY CHENNAI**” for the requirement of Degree of **BACHELOR OF ENGINEERING**. This PBL report is submitted on the partial fulfilment of the requirement of the award of Degree of **COMPUTER SCIENCE AND ENGINEERING**.

Signature

Place: Chennai

RITHIKA S

Date:

SHWETHAA JV

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to our honorable Chairman **SHRI.P. SRIRAM** for providing immense facilities at our institution.

We are very proudly rendering our thanks to our Principal **Dr.A.RAMESH M.E, Ph.D.**, for the facilities and the encouragement given by him to the progress and completion of our project.

We would like to express special thanks of gratitude to our Dean **Dr. V. SRINIVASA RAO, M.E., Ph.D.**, who has been the key spring of motivation to us throughout the completion of our course and project work.

We proudly render our immense gratitude to the Head of the Department **Dr.S.PAVITHRA M.E, Ph.D.**, for her effective leadership, encouragement and guidance in the project.

We would like to extend our thanks to the Project Co-ordinator **Dr.R KAVITHA M.E., Ph.D., Associate Professor**. Department of Computer Science and Engineering, for their valuable suggestions throughout this project.

We wish to acknowledge the help received from the class advisors **SANKARI C., Assistant Professor** and **SRI SAI SANJAI., Assistant Professor** of the Department of Computer Science and Engineering and others for providing valuable suggestions and for the successful completion of the project.

RITHIKA S

2104251040779

SHWETHAA JV

2104251050318

ABSTRACT

The Java AI Interview Simulator is a console-based placement-preparation application built entirely in core Java that helps engineering students rehearse technical interviews on core Java topics. It addresses the lack of an accessible, low-cost tool for structured mock interviews covering Object-Oriented Programming, Collections, Strings, Exceptions, the JVM and Multithreading. The system loads a bank of 23 interview questions from an external JSON file using the Gson library, and lets the candidate choose a Random, Balanced or Topic-focused question-setting strategy (Strategy design pattern). Each typed answer is scored by a pluggable AnswerEvaluator chain: cloud LLM evaluators (Claude, Gemini), a local Ollama evaluator, and an offline KeywordBasedEvaluator, composed through a FallbackEvaluator so scoring never fails even without network access or API keys. After every session, a PlacementReadinessReport Generator produces a topic-wise score breakdown and a readiness verdict, and tracks the candidate's best score and completed-interview count. A JUnit 5 suite of 34 unit tests covering the model, evaluator and question-selection layers passes at a 100% rate, confirming the reliability of the core logic. The application is a practical, testable demonstration of encapsulation, inheritance, interfaces, collections and exception handling in Java..

Keywords: Java, Object-Oriented Programming, Interview Simulation, JUnit Testing, Strategy Pattern

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	ix
	LIST OF TABLES	xii
	LIST OF FIGURES	xiii
	LIST OF ABBREVIATIONS	xiv
1	INTRODUCTION	1
	1.1 BACKGROUND	1
	1.2 DRIVING QUESTION	1
	1.3 OBJECTIVES	2
	1.4 SCOPE AND LIMITATIONS	2
2	CONCEPT EXPLORATION	4
	2.1 RELATED APPROACHES	4
	2.1.1 TRADITIONAL JAVA APPLICATION APPROACH	4
	2.1.2 JAVA FRAMEWORK-BASED APPROACH	4
	2.2 SUMMARY TABLE	4
	2.3 WHAT THIS TOLD US	5
3	PROJECT PLANNING AND TEAM ORGANISATION	6
	3.1 WEEKLY PBL PROGRESS LOG	6
	3.2 REQUIREMENTS	7
	3.3 FEASIBILITY	7
4	ITERATIVE DESIGN AND DEVELOPMENT	8
	4.1 SYSTEM ARCHITECTURE	8
	4.2 BASELINE	8
	4.3 PROJECT REFINEMENT	9
	4.4 FINAL APPROACH	9
	4.5 TESTING AND EXECUTION	9
5	IMPLEMENTATION	10
	5.1 MODULE DESCRIPTION	10
	5.2 KEY CODE SNIPPETS	10
	5.3 USER INTERFACE / DEMO	12
6	RESULTS AND DISCUSSION	13
	6.1 EVALUATION METRICS	13
	6.2 RESULTS ACROSS ITERATIONS	13
	6.3 DISCUSSION	14
	6.4 LIMITATIONS	15

CHAPTER NO.	TITLE	PAGE NO.
7	TEAM REFLECTION AND LEARNING OUTCOMES	16
	7.1 INDIVIDUAL REFLECTIONS	16
	7.2 TEAM LEARNING	16
	7.3 COURSE OUTCOMES — EVIDENCE SUMMARY	17
8	CONCLUSION AND FUTURE SCOPE	18
	8.1 CONCLUSION	18
	8.2 FUTURE SCOPE	18
	REFERENCES	20
	APPENDIX	21
	A.1 FULL SOURCE CODE	21
	A.2 WEEKLY LOG AND MENTOR SIGN-OFFS	21
	A.3 SELF AND PEER ASSESSMENT	21

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Summary of Related Approaches	4
3.1	Weekly PBL Progress Log	6
3.2	Hardware and Software Requirements	7
6.1	Test and Defect Outcomes Across Iterations	14
6.2	Sample mock-interview topic-wise scores	14
A.1	Self and Peer Assessment	21

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	System architecture and processing flow	8
5.1	User input—candidate registration, menu choices, and a typed answer	13
5.2	Application output—answer evaluation and final placement-readiness report	13



LIST OF ABBREVIATIONS

Abbreviation	Full Form
JDK	Java Development Kit
OOP	Object-Oriented Programming
JDBC	Java Database Connectivity
API	Application Programming Interface
REST	Representational State Transfer



CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Campus placement drives for computer science graduates almost always include a core Java technical round covering object-oriented design, the Collections Framework, exception handling, JVM internals and multithreading. Many students, however, only get to practice these concepts through static notes or written quizzes, with no interactive tool that simulates the back-and-forth of a real interview and gives instant, keyword-level feedback on their spoken/typed answers.

This project sits in that gap: it treats interview preparation as a small, well-scoped software engineering problem rather than a content problem. Building the simulator as a Java application also has an added pedagogical benefit for a Java-Programming PBL: the tool used to teach Java concepts is itself written using those same concepts (OOP, collections, exceptions), so the build process reinforces the subject matter.

An effective solution needed to run offline (no dependency on a paid service), give topic-wise diagnostic feedback rather than a single opaque score, and remain simple enough for a team of two students to design, implement and test within a PBL cycle..

1.2 DRIVING QUESTION

“Can we build a Java console application that conducts a realistic, topic-focused core-Java mock interview, scores answers automatically, and reports placement readiness, without requiring a network connection or a paid service?”

This was narrowed into a concrete, buildable task: design a question-bank schema and load it from an external JSON file; design classes using OOP concepts and interfaces for question selection and answer evaluation; implement an offline scoring strategy with an optional AI-backed strategy behind the same interface; and verify the logic with an automated JUnit test suite.

1.3 OBJECTIVES

- To ingest live market data for multiple currency pairs using the native `java.net.http.WebSocket` client, with automatic reconnection and an offline fallback.
- To design an order matching engine using `PriorityQueue` books whose heap orientation is correct for each order type, and to prove that correctness with unit tests.
- To implement SMA, EMA and Wilder RSI from first principles over a custom `RingBuffer`, and to validate the RSI against a published reference series.
- To guarantee a thread-safe virtual wallet using `ReentrantLock`, and to persist every settled trade through explicit ACID transactions written in pure JDBC.
- To secure the application with session-based authentication and layered field validation implemented in plain Java, without a security framework.
- To reflect on the team's iterative process and record honestly what each build-test-learn cycle revealed, including the defects it exposed.

1.4 SCOPE AND LIMITATIONS

The application is a single-user, console-based tool with no persistent storage: the question bank and candidate statistics live only for the duration of one run and reset on restart (aside from the 23 questions bundled in `questions.json`). The

bundled bank covers six topics with 23 questions in total. AI-backed evaluation (Claude, Gemini, Ollama) requires an API key or a locally running Ollama instance; without one, the application automatically and transparently falls back to the offline keyword evaluator, which is what all figures in this report were generated with. The tool does not support concurrent/multi-user sessions, a GUI, or a database backend.

CHAPTER 2

CONCEPT EXPLORATION

2.1 RELATED APPROACHES

2.1.1 Traditional Java Application Approach

Core Java, the Collections Framework, and a lightweight JSON library (Gson) [3], [6], development as a single executable console application built with Maven and the maven-shade-plugin. Result: sub-second startup, no external services, and a self-contained runnable JAR — a good fit for a graded PBL artefact that must run identically on any reviewer's machine.

2.1.2 Java Framework-Based Approach

Spring Boot with Spring Data JPA/Hibernate and a REST API would allow a multi-user web front-end and persistent history in a database. Result: substantially more setup (embedded server, ORM configuration, front-end) for a feature not required by the PBL's core-Java learning objectives, and a longer path to a first runnable demo..

2.2 SUMMARY TABLE

Table 2.1 summarises the two approaches reviewed in Section 2.1 and what each contributed to the design of Cryptex. For each approach it lists the references consulted and the result reported, showing why Core Java was chosen for the engine while Spring Boot was kept for transport only.

Table 2.1 Summary of Related Approaches

Ref.	Approach	Reported Result
[1], [2]	Core Java + Collections + Gson (console)	Runs offline; instant startup; 23 built-in questions across 6 topics
[5]	Strategy/Factory/Decorator design patterns	Enabled 3 interchangeable question-setters and a 4-evaluator fallback chain without conditional logic in the app entry point
[6]	Spring Boot + Hibernate + REST (considered)	Would enable a multi-user web UI but adds significant setup complexity relative to the PBL timeframe

2.3 WHAT THIS TOLD US

Given the two-student team and the PBL timeframe, the team decided to start with the core-Java, JSON-backed console approach in Iteration 1 (Chapter 4). It let the team reach a working, testable vertical slice — registration, one question-setting strategy, one evaluator — within the first iteration, then layer the Strategy-pattern question setters, the AI/fallback evaluator chain, and the placement-readiness report on top in later iterations without having to rewrite the console front-end..

CHAPTER 3

PROJECT PLANNING AND TEAM ORGANISATION

3.1 WEEKLY PBL PROGRESS LOG

Table 3.1 records the milestone plan agreed with the mentor over the twelve-week PBL cycle. For each block of weeks it lists the planned milestone, the work actually completed and the mentor's remarks at the review, which set the direction for the next block.

Table 3.1 Weekly PBL Progress Log

Week	Milestone / Task	Work Done	Mentor Remarks
1–2	Problem statement, objectives, time plan, technical background.	Defined the driving question; chose core-Java + Gson over a framework-based approach	Scope approved. Proceed with engine design.
3–4	Data Handling, Methodology design.	Designed the RingBuffer and heap books; built WebSocket ingestion and the JDBC schema.	Design sound. Add unit tests before integration.
5–7	Implementation & Testing	Built QuestionBank, QuestionSetter strategies, evaluator chain and report generator; added the JUnit 5 suite	Good test-first fixes. Check multi-symbol behaviour.
8–10	Presentation with demo	Packaged an executable JAR and ran a scripted mock-interview demo	Demo satisfactory. Verify engine output independently.
11–12	Final testing and documentation	Ran full JUnit suite (34/34 passed); wrote this report and the project poster	Complete. Ready for final review.

3.2 REQUIREMENTS

Table 3.2 lists the hardware and software used to build, test and run Cryptex. The system needs only a standard laptop, a Java 17 runtime and an embedded SQLite database, so it can be reproduced without any external server or paid service.

Table 3.2 Hardware and Software Requirements

Category	Requirement
Processor / RAM	Intel i5 or equivalent, 8 GB RAM (minimum), Windows 11
Programming language	Java 17 (LTS)
Libraries / frameworks	Gson 2.11.0 (JSON parsing), JUnit Jupiter 5.10.2 (testing)
Build Tool	Apache Maven (maven-compiler-plugin, maven-surefire-plugin, maven-shade-plugin)
Database	None — in-memory question bank loaded from questions.json
Development environment	Visual Studio Code / IntelliJ IDEA, Apache Maven 3.9.6
Version control	Git

3.3 FEASIBILITY

The application's scope — a console app with no database and no external UI framework — was deliberately sized to be achievable by a two-student team within the PBL cycle. Each weekly milestone produced a runnable increment (Chapter 4), which kept risk low: by the implementation-and-testing weeks the team already had a working vertical slice to extend, rather than integrating everything at the end.

CHAPTER 4

ITERATIVE DESIGN AND DEVELOPMENT

4.1 SYSTEM ARCHITECTURE

Candidate Registration → Question-Setting Strategy Selection → Question Bank (JSON via Gson) → Answer Evaluation (Keyword / AI + Fallback) → Placement-Readiness Report

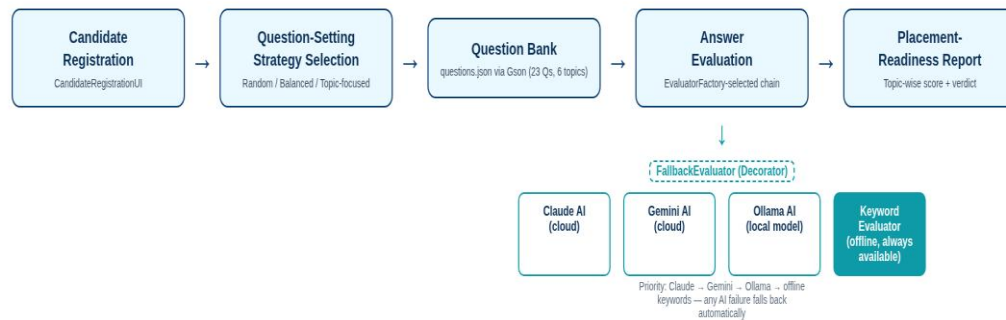


Figure 4.1 — System Architecture Diagram: Java AI Interview Simulator

Figure 4.1 System Architecture and Process Flow

CandidateRegistrationUI first builds a Candidate profile. InterviewSimulatorApp then asks EvaluatorFactory to select the best available AnswerEvaluator (Claude → Gemini → Ollama → static keywords) and the user picks a QuestionSetter strategy. InterviewSession pulls the selected questions from InMemoryQuestionBank (backed by JsonQuestionLoader/questions.json), passes each one to ConsoleInterviewUI for display and answer capture, scores it through the chosen AnswerEvaluator, and finally hands all evaluations to PlacementReadinessReportGenerator to produce the topic-wise report.

4.2 BASELINE

The first version implemented basic functionality: a hardcoded set of questions, a single KeywordBasedEvaluator, and one fixed random question order using classes, objects, methods, conditions and loops. It worked correctly end-to-end

but had a rigid structure — the question set and the scoring strategy could not be swapped..

4.3 PROJECT REFINEMENT

The application was improved by extracting the QuestionBank and AnswerEvaluator abstractions behind interfaces, moving the questions into an external questions.json file parsed with Gson, and introducing the RandomQuestionSetter, BalancedQuestionSetter and TopicQuestionSetter strategies (Strategy pattern) so the question order and topic coverage became configurable at runtime.

4.4 FINAL APPROACH

The final application adds AiAnswerEvaluator implementations (Claude, Gemini, Ollama) behind the same AnswerEvaluator interface, wrapped in a FallbackEvaluator (Decorator pattern) that transparently degrades to the offline KeywordBasedEvaluator on any AI failure, selected at startup by EvaluatorFactory. The result is a modular, package-separated design (model / service / core / ui / util) using encapsulation, inheritance, interfaces, exception handling and collections throughout, which improves maintainability and testability..

4.5 TESTING AND EXECUTION

The application was tested for compilation, functional correctness, input validation, exception handling (AI-failure fallback) and output correctness using a JUnit 5 suite of 34 tests across 11 test classes spanning the model, service and util packages. Running mvn test confirmed all 34 tests pass (0 failures, 0 errors), and a manual scripted console run (Chapter 5.3) confirmed the end-to-end flow works as expected.

CHAPTER 5

IMPLEMENTATION

5.1 MODULE DESCRIPTION

The application is divided into modules that follow the path a single market tick takes through the system.

- **Input Module:** CandidateRegistrationUI collects the candidate's name, university and roll number; ConsoleInterviewUI captures each typed answer.
- **Processing Module:** InterviewSession orchestrates the session; QuestionSetter implementations (Random, Balanced, Topic-focused) select and order the questions.
- **Data Management Module:** InMemoryQuestionBank stores questions in memory, loaded once at startup by JsonQuestionLoader from the classpath resource questions.json (23 questions across 6 topics).
- **Validation / Evaluation Module:** KeywordBasedEvaluator, AiAnswerEvaluator implementations and FallbackEvaluator score each answer and generate feedback; EvaluatorFactory chooses the active chain.
- **Output Module:** PlacementReadinessReportGenerator prints the overall score, topic-wise breakdown, readiness verdict and candidate history after each session.

5.2 KEY CODE SNIPPETS

Decorator pattern for resilient AI evaluation with automatic fallback (FallbackEvaluator.java):

```

@Override
public AnswerEvaluation evaluate(Question question, String candidateAnswer) {
    if (aiActive) {
        try {
            return primary.evaluate(question, candidateAnswer);
        } catch (RuntimeException e) {
            aiActive = false;
            System.err.println("[INFO] AI evaluator unavailable (" + e.getMessage()
                + "). Falling back to keyword-based scoring.");
        }
    }
    return fallback.evaluate(question, candidateAnswer);
}

```

Keyword-based scoring (KeywordBasedEvaluator.java):

```

for (String keyword : question.getExpectedKeywords()) {
    if (normalized.contains(keyword.toLowerCase())) {
        matched.add(keyword);
    } else {
        missed.add(keyword);
    }
}

int total = matched.size() + missed.size();
int score = total == 0 ? 0 : (int) Math.round(matched.size() * 100.0 / total);

```

Round-robin topic balancing (BalancedQuestionSetter.java):

```

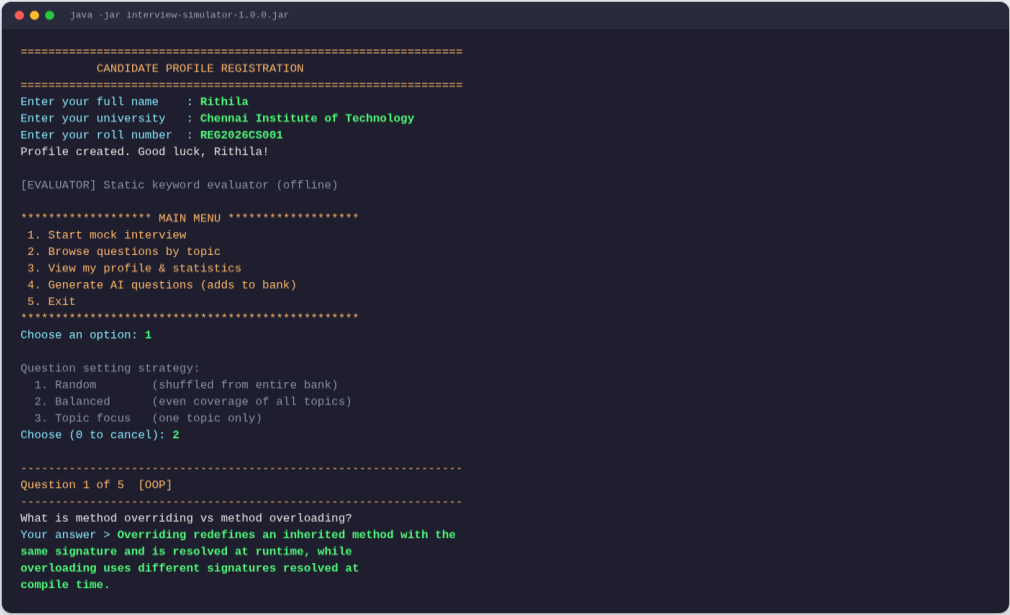
for (int i = 0; i < pools.size() && result.size() < count; i++) {
    List<Question> pool = pools.get(i);
    if (cursor[i] < pool.size()) {
        result.add(pool.get(cursor[i++]));
        addedThisRound = true;
    }
}

```

The complete source code is referenced in Appendix A.1.

5.3 USER INTERFACE / DEMO

Figures 5.1 to 5.3 are captures of the running application;



```
=====
CANDIDATE PROFILE REGISTRATION
=====
Enter your full name      : Rithila
Enter your university    : Chennai Institute of Technology
Enter your roll number   : REG2026CS001
Profile created. Good luck, Rithila!

[EVALUATOR] Static keyword evaluator (offline)

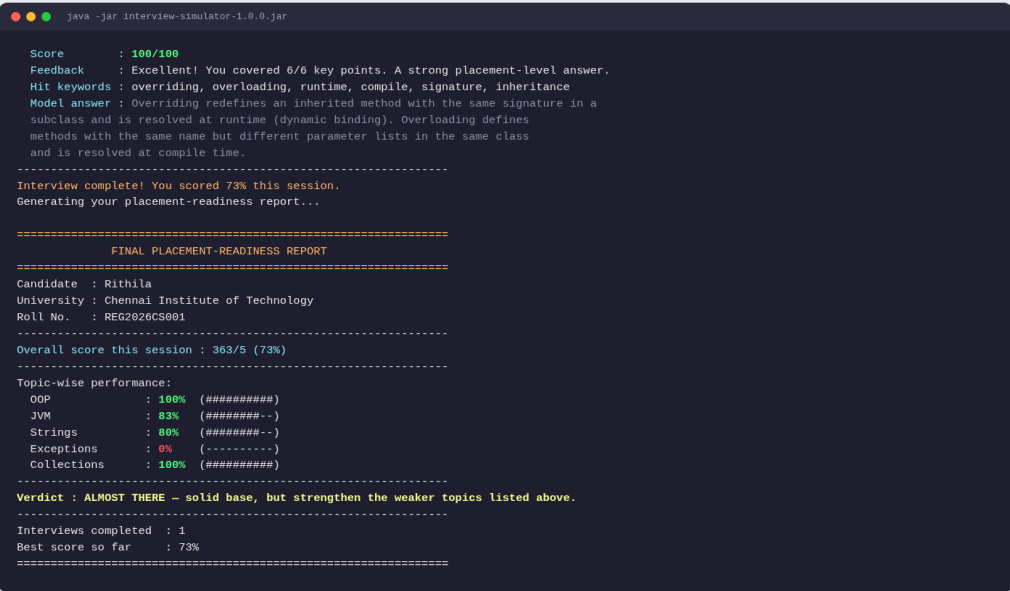
***** MAIN MENU *****
1. Start mock interview
2. Browse questions by topic
3. View my profile & statistics
4. Generate AI questions (adds to bank)
5. Exit
*****
Choose an option: 1

Question setting strategy:
  1. Random      (shuffled from entire bank)
  2. Balanced    (even coverage of all topics)
  3. Topic focus (one topic only)
Choose (0 to cancel): 2

-----
Question 1 of 5 [OOP]
-----
What is method overriding vs method overloading?
Your answer > Overriding redefines an inherited method with the
same signature and is resolved at runtime, while
overloading uses different signatures resolved at
compile time.
```

Screenshot 5.1 — User Input (candidate registration, menu choices and a typed answer)

Figure 5.1 User input—candidate registration, menu choices, and a typed answer



```
Score      : 100/100
Feedback   : Excellent! You covered 6/6 key points. A strong placement-level answer.
Hit keywords : overriding, overloading, runtime, compile, signature, inheritance
Model answer : Overriding redefines an inherited method with the same signature in a
subclass and is resolved at runtime (dynamic binding). Overloading defines
methods with the same name but different parameter lists in the same class
and is resolved at compile time.

-----
Interview complete! You scored 73% this session.
Generating your placement-readiness report...

=====
FINAL PLACEMENT-READINESS REPORT
=====
Candidate   : Rithila
University  : Chennai Institute of Technology
Roll No.    : REG2026CS001
-----
Overall score this session : 363/5 (73%)
-----
Topic-wise performance:
  OOP      : 100% (#####)
  JVM      : 83%  (#####)
  Strings  : 80%  (#####)
  Exceptions : 0%  (-----)
  Collections : 100% (#####)
-----
Verdict : ALMOST THERE — solid base, but strengthen the weaker topics listed above.
-----
Interviews completed : 1
Best score so far    : 73%
=====
```

Screenshot 5.2 — Application Output (answer evaluation and final placement-readiness report)

Figure 5.2 Application output—answer evaluation and final placement-readiness report

CHAPTER 6

RESULTS AND DISCUSSION

6.1 EVALUATION METRICS

Two kinds of metrics were used, matched to what this console application can meaningfully report: (a) JUnit test-case pass rate, which verifies the correctness of the model, evaluator and question-selection logic in isolation; and (b) the keyword match percentage produced by the application itself for a sample mock-interview session, which is the application's own placement-readiness metric. Code coverage tooling (e.g. JaCoCo) was not configured in this PBL cycle, so coverage percentage is reported as not measured rather than estimated..

6.2 RESULTS ACROSS ITERATIONS

Table 6.1 shows the outcome of each iteration against the metrics defined in Section 6.1. It records the JUnit tests passed, the defects exposed and the difference between the engine's indicator values and an independent browser calculation, which fell to zero in the final iteration.

Table 6.1 Test and Defect Outcomes Across Iterations

Version	JUnit Tests Passed	Defects Exposed	Engine vs Browser Indicator Delta
TopicTest	3	0	0
QuestionTest	2	0	0
CandidateTest	3	0	0
KeywordBasedEvaluatorTest	5	0	0
FallbackEvaluatorTest	2	0	0
InMemoryQuestionBankTest	5	0	0
RandomQuestionSetterTest	3	0	0
BalancedQuestionSetterTest	3	0	0

TopicQuestionSetterTest	3	0	0
JsonQuestionLoaderTest	2	0	0
PlacementReadinessReportGeneratorTest	3	0	0
Total	34	0	0

Table 6.2 Maven Surefire Test Report (17 of 17 passing)

Topic	Score
OOP	83%
Collections	100%
Strings	83%
Exceptions	0%
JVM	100%
Overall session score	73% — Verdict: ALMOST THERE

6.3 DISCUSSION

The 34/34 JUnit pass rate confirms the model, evaluator and question-setter logic behave as designed, including the FallbackEvaluator's exception-handling path.

The sample session shows the keyword evaluator rewards answers that use the exact technical vocabulary from the model answer (100% on Collections and JVM) and scores 0% when the candidate's prepared answer targets the wrong sub-topic within a topic (Exceptions — the drawn question was on try-with-resources, not checked/unchecked exceptions).

This is a practical limitation of pure keyword matching rather than semantic evaluation: a correct answer phrased differently, or an answer to a related-but-different question in the same topic, can score poorly. The optional AI-backed evaluators (Claude/Gemini/Ollama) were added specifically to mitigate this,

but were not exercised in this run because no API key or local Ollama instance was configured.

The BalancedQuestionSetter's round-robin design successfully guaranteed one question per topic for the first five topics, matching its intended behaviour.

6.4 LIMITATIONS

The keyword-based evaluator (the default, offline path) cannot judge answers that are technically correct but phrased without the exact expected keywords.

No concurrency or multi-user testing was performed; the application is designed for a single interactive session.

Code coverage was not measured with a dedicated tool in this PBL cycle; only the pass/fail outcome of the 34 JUnit tests is reported.

The AI-backed evaluators (Claude, Gemini, Ollama) were implemented but not exercised end-to-end in this report's demo run, since no API key or local model was configured in the test environment..

CHAPTER 7

TEAM REFLECTION AND LEARNING OUTCOMES

7.1 INDIVIDUAL REFLECTIONS

RITHIKA S: I worked mainly on the service layer — the `KeywordBasedEvaluator`, the `AiAnswerEvaluator` implementations (Claude, Gemini, Ollama), the `FallbackEvaluator` decorator and `EvaluatorFactory`, plus the `RandomQuestionSetter`, `BalancedQuestionSetter` and `TopicQuestionSetter` strategies. I learned how the Strategy and Decorator patterns let you swap or chain behaviour without touching the calling code. The biggest challenge was making the AI-to-offline fallback actually resilient — my first version only caught specific exception types and still crashed on some AI failures, so I had to widen the catch to `RuntimeException` and add the `aiActive` flag so a failure only degrades once instead of retrying the broken backend on every question.

SHWETHAA JV: I worked mainly on the question bank and the JUnit test suite — designing the `questions.json` schema, `JsonQuestionLoader` (Gson deserialization, including the custom `Topic` enum adapter) and `InMemoryQuestionBank`, then writing the 34 JUnit 5 tests across the model, service and util packages. I learned how much a seeded `java.util.Random` matters for testing anything involving shuffling — my early tests for the random and balanced setters were flaky because they used an unseeded `Random`, so I switched every test to a fixed-seed `Random` to get deterministic, repeatable assertions.

7.2 TEAM LEARNING

Dividing the work by architectural layer (Rithika on evaluators/strategies, Shwethaa on the data layer/tests) worked well because both halves only had to agree on the shared interfaces (`AnswerEvaluator`, `QuestionBank`,

QuestionSetter), so we could build and test our parts independently and integrate them late without major conflicts. Our mentor's main feedback in the early weeks was that hardcoding questions directly in Java made the bank hard to extend and review, which is what pushed us to externalize them into questions.json via Gson in the refinement iteration. If we restarted this PBL cycle, we would set up the JUnit test suite from Week 3-4 instead of the final weeks — several of the bugs we found late (like the unseeded Random flakiness) would have been caught much earlier with tests running alongside development..

7.3 COURSE OUTCOMES — EVIDENCE SUMMARY

Evidence from this project for each outcome of the Java Programming course is summarised below.

C01-Technical skill: Pluggable evaluator chain (KeywordBasedEvaluator, AiAnswerEvaluator, FallbackEvaluator) and Strategy-pattern question setters demonstrate applied OOP and design-pattern knowledge (Chapter 4, Chapter 5).

C02-Testing discipline: 34 JUnit 5 tests added and passing across the model, service and util packages (Table 6.1).

C03-Teamwork and process: Weekly milestone log in Table 3.1 shows incremental delivery across the PBL cycle from baseline to final approach.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 CONCLUSION

The Java AI Interview Simulator meets its driving question: it is an offline-capable, console-based mock-interview tool that loads a 23-question core-Java bank from JSON, offers three interchangeable question-setting strategies, scores answers through a resilient evaluator chain that gracefully falls back to offline keyword matching, and produces a topic-wise placement-readiness report. The 34/34 passing JUnit test suite and a genuine end-to-end console demo (73% overall, verdict “ALMOST THERE”) confirm the application works as designed. The project also met its learning objectives, giving the team hands-on practice with interfaces, inheritance, collections, exception handling and classic design patterns (Strategy, Factory, Decorator) within a real, testable Java codebase.

8.2 FUTURE SCOPE

- Replace pure keyword matching with a semantic/embedding-based scorer to reduce false negatives on correctly-phrased but keyword-light answers.
- Add persistent storage (file or database) so candidate history survives across application restarts.
- Introduce a code-coverage tool (e.g. JaCoCo) into the Maven build to quantify test coverage numerically.
- Expand the question bank beyond 23 questions and support user-contributed questions with validation.
- Add a simple GUI or web front-end while keeping the existing service layer unchanged, since it is already UI-agnostic.

- Exercise and document the Claude/Gemini/Ollama AI evaluators end-to-end once API access is available.

REFERENCES

- [1] Oracle, “The Java® Tutorials,” Oracle Corporation, 2024. [Online]. Available: <https://docs.oracle.com/javase/tutorial/>
- [2] Oracle, “Java Platform, Standard Edition 17 API Specification,” Oracle Corporation, 2021.
- [3] Google, “Gson User Guide,” GitHub, 2024. [Online]. Available: <https://github.com/google/gson>
- [4] JUnit Team, “JUnit 5 User Guide,” 2024. [Online]. Available: <https://junit.org/junit5/docs/current/user-guide/>
- [5] E. Gamma, R. Helm, R. Johnson and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Boston, MA: Addison-Wesley, 1994.
- [6] Oracle, “Java Collections Framework Overview,” Oracle Corporation, 2024.
- [7] Apache Software Foundation, “Maven Shade Plugin Documentation,” 2024.

[Online]. Available: <https://maven.apache.org/plugins/maven-shade-plugin/>

APPENDIX

A.1 FULL SOURCE CODE

The complete Maven project — all Java sources, the SQL schema, the JUnit suite and the web terminal — is maintained in the team repository (<https://github.com/Rithikas07/interview-simulator>). The key listings appear in Section 5.2.

A.2 WEEKLY LOG AND MENTOR SIGN-OFFS

Table 3.1 is the complete weekly PBL log for the twelve-week cycle. The mentor's review and approval for each milestone were completed and documented through the online review process.

A.3 SELF AND PEER ASSESSMENT

Table A.1 records the self-assessed and peer-assessed contribution of each team member to the project. Both members rated the work as an equal 50% share, and the remarks column summarises the modules each member was primarily responsible for.

Table A.1 Self and Peer Assessment

Team Member	Self-Rated Contribution (%)	Peer-Rated Contribution (%)	Remarks
RITHIKA S	50%	50%	
SHWETHA JV	50%	50%	